



Interface in Java

One
Interface
Many
Possibilities

Define | Implement | Achieve Abstraction | Build Flexible Code



What is Interface?

- An interface in Java is a reference type, similar to a class, that contains only abstract methods (methods without a body).
- It is used to achieve abstraction and multiple inheritance in Java.
- A class implements an interface to provide the implementation of its methods.
- An interface cannot be instantiated (we cannot create its object).



Syntax

```
interface InterfaceName
{
    // abstract method (no body)
    returnType methodName();

    // constant (implicitly public static final)
    type CONSTANT_NAME = value;
}
```



Real-World Analogy



An interface is like a power socket. It defines a standard, but the actual device (phone, laptop, etc.) provides its own implementation to use that standard.



Why Use Interface?

- ✓ Achieve abstraction
- ✓ Support multiple inheritance
- ✓ Define a common contract for different classes
- ✓ Promote loose coupling (flexible and maintainable code)
- ✓ Widely used in frameworks (e.g., Spring, Hibernate, etc.)



Key Points

- 1 Declared using the keyword **interface**.
- 2 Contains abstract methods (by default public and abstract).
- 3 Can have variables (by default public, static and final).
- 4 A class implements an interface using the **implements** keyword.
- 5 Supports multiple inheritance.
- 6 Used to achieve abstraction.
- 7 Introduced in Java 1.0 and enhanced in later versions (default and static methods in Java 8).



Example

```
// Interface
interface Animal
{
    void sound(); // abstract method
}

// Class implementing interface
class Dog implements Animal
{
    public void sound()
    {
        System.out.println("Dog barks");
    }
}
```

Output:
Dog barks



Class vs Interface

Feature	Class	Interface
Keyword	class	interface
Methods	Can have concrete and abstract methods	Only abstract methods (by default)
Variables	Can have any type	Only public static final
Constructor	Yes	No
Inheritance	Single inheritance	Multiple inheritance (via multiple interfaces)
Object Creation	Possible	Not possible



Same Contract
Different Implementations
Greater Possibilities



LEARN INTERFACE | CODE BETTER | BUILD FLEXIBLE SOLUTIONS

Introduction to Interface

An interface in Java is a blueprint that defines a set of methods that a class must implement. Interfaces are mainly used to achieve abstraction, loose coupling, multiple inheritance, and standardization of behaviour.

An Interface in Java is a blueprint of a class that contains behavior (methods) but generally does not contain implementation.

An interface tells a class:

"These are the methods you MUST implement."

Interfaces also support multiple inheritance in Java.

- ✓ A class must implement all abstract methods of an interface.
- ✓ All variables in an interface are public, static, and final by default.
- ✓ Interfaces can have default, static, and private methods (Java 8+ and 9+).

An interface is declared using the `interface` keyword.

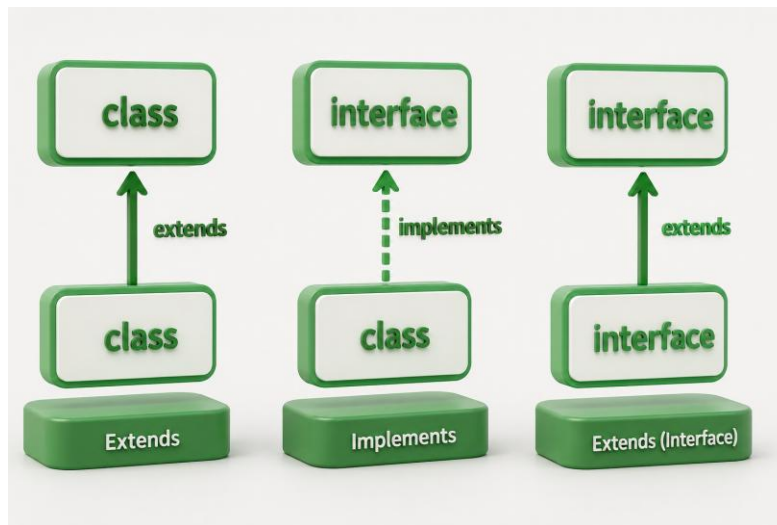
```
interface Animal
{
    void sound();
}
```

Here:

- `Animal` is an interface.
- `sound()` is an abstract method.
- The method does not contain a body.
- A class implements an interface using the `implements` keyword.

Relationship Between Class and Interface

A class can extend another class and similarly, an interface can extend another interface. However, only a class can implement an interface and the reverse (an interface implementing a class) is not allowed.



Use Case – Class

- ✓ Use a class when you need to represent a real-world entity with attributes (fields) and behaviors (methods).
- ✓ Use a class when you need to create objects that hold state and perform actions
- ✓ Classes are used for defining templates for objects with specific functionality and properties.

Use Case – Interface

- ✓ Use an interface when you need to define a contract for behavior that multiple classes can implement.
- ✓ Interface is ideal for achieving abstraction and multiple inheritance.

Why Do We Need Interfaces?

Suppose we have different payment methods:



All payment methods perform a `pay()` operation, but their implementation is different.

An interface can define the common behavior:

```
interface Payment
{
    void pay();
}
```

Then different classes provide their own implementation.

```
class UPI implements Payment
{
    @Override
    public void pay()
    {
        System.out.println("Payment through UPI");
    }
}
```

```
class CreditCard implements Payment
{
    @Override
    public void pay()
    {
        System.out.println("Payment through Credit Card");
    }
}
```

Now the program can work with the common Payment interface.

Basic Syntax:

```
interface InterfaceName
{
    // constants
    // abstract methods
    // default methods
    // static methods
    // private methods
}
```

- ✓ All the methods are by default abstract.
- ✓ You can define a default method in an interface.
- ✓ You can create a static method also.
- ✓ You can create a private method.

A class implements it using:

```
class ClassName implements InterfaceName
{
    // implementation
}
```

Interface Rules

1. Cannot create object.

```
Payment p = new Payment();
```

Error:

Payment is abstract; cannot be instantiated

2. A class must implement all methods

```
interface Animal
{
    void eat();
    void sleep();
}
```

3. Methods are automatically

```
public abstract
interface Demo
{
    void show();
}
```

Actually means:

```
public abstract void show();
```

4. Variables are automatically

```
public static final
```

Example:

```
interface Demo
{
    int AGE=20;
}
```

Actually means:

```
public static final int AGE=20;
```

Example: 1

```
interface Animal
{
    void sound();
}
class Dog implements Animal
{
    public void sound()
    {
        System.out.println("Dog Barks");
    }
    public static void main(String args[])
    {
        Dog d = new Dog();
        d.sound();
    }
}
```

Example: 2

```
interface Payment
{
    void pay(double amount);
}
//Credit Card
class CreditCardPayment implements Payment
{
    public void pay(double amount)
    {

```

```

        System.out.println("Paid Rs. " + amount + " using Credit
                                Card");
    }
}
//UPI
class UIPayment implements Payment
{
    public void pay(double amount)
    {
        System.out.println("Paid Rs. " + amount + " using UPI");
    }
}
//PayPal
class PaypalPayment implements Payment
{
    public void pay(double amount)
    {
        System.out.println("Paid Rs. " + amount + " using Paypal");
    }
}
public class PaymentSystem
{
    public static void main(String args[])
    {
        Payment p1 = new CreditCardPayment();    //Interface Reference
        Payment p2 = new UIPayment();
        Payment p3 = new PaypalPayment();
        p1.pay (1000);
        p2.pay (500);
        p3.pay (3000);
    }
}

```

```
}
```

Multiple Interfaces

This is one of the most important advantages of interfaces. Java doesn't support multiple inheritance using classes.

But interfaces solve that problem.

Example:

```
interface Camera
{
    void click();
}
interface Music
{
    void playMusic();
}
class Smartphone implements Camera, Music
{
    public void click()
    {
        System.out.println("Photo Taken");
    }
    public void playMusic()
    {
        System.out.println("Playing Music");
    }
    public static void main(String[] args)
    {
```

```
Smartphone phone = new Smartphone();  
phone.click();  
phone.playMusic();  
}  
}
```

OUTPUT
Photo Taken
Playing Music

Interface Extending Another Interface

An interface can extend another interface using **extends**.

Example:

```
interface Animal  
{  
    void eat();  
}  
//Another interface:  
interface Dog extends Animal  
{  
    void bark();  
}  
//A class implementing Dog must implement both methods:  
class InterfaceInheritance implements Dog  
{  
    @Override  
    public void eat()  
    {  
        System.out.println("Dog eats");  
    }  
    @Override  
    public void bark()  
    {  
        System.out.println("Dog barks");  
    }  
}
```

```
}  
public static void main(String[] args)  
{  
    InterfaceInheritance dog = new InterfaceInheritance();  
    dog.eat();  
    dog.bark();  
}  
}
```

OUTPUT
Dog eats
Dog barks

Important:

An interface can extend multiple interfaces.

```
interface A  
{  
    void methodA();  
}  
  
interface B  
{  
    void methodB();  
}  
  
interface C extends A, B  
{  
    void methodC();  
}
```

A class implementing C must implement all three methods.

Methods in Interface:

Default Method

Starting with Java 8, interfaces can contain default methods with a method body.

Example:

```
interface Vehicle
{
    void start();
    default void stop()
    {
        System.out.println("Vehicle Stopped");
    }
}

class DefaultMethod implements Vehicle
{
    @Override
    public void start()
    {
        System.out.println("Car Started");
    }

    public static void main(String[] args)
    {
        DefaultMethod dm = new DefaultMethod();
        dm.start();
        dm.stop();
    }
}
```

OUTPUT

```
Car Started
Vehicle Stopped
```

The Car class does not have to override `stop()` because the interface already provides its default implementation.

Static Method

Static methods in Java interfaces (introduced in Java 8) are utility methods that belong to the interface itself, not to any implementing class. They must be called using the interface name and cannot be overridden.

Key Characteristics:

- ✓ **Definition:** Declared with the `static` keyword inside an interface.
- ✓ **Implementation:** Must have a complete method body (unlike abstract methods).
- ✓ **Access:** Called using the interface name, e.g., `InterfaceName.methodName()`.

Not

```
object.methodName(); //Wrong Calling
```

- ✓ **Inheritance:** Not inherited by implementing classes or sub-interfaces.
- ✓ **Overriding:** Cannot be overridden by implementing classes.
- ✓ **Purpose:** Best suited for utility methods, factory methods, or validators that logically belong to the interface.

Example:

```
interface Calculator
{
    static int square (int n)
    {
        return n * n;
    }
}
```

```

}
public class InterfaceStaticMethod
{
    public static void main (String[] args)
    {
        int result = Calculator.square(5);
        System.out.println(result);
    }
}

```

Private Methods

Since Java 9, interfaces can contain private methods.

Private methods are useful for sharing implementation code between default and static methods.

```

interface Vehicle
{
    default void start()
    {
        log("Vehicle is starting");
    }

    default void stop()
    {
        log("Vehicle is stopping");
    }

    private void log(String message)
    {
        System.out.println(message);
    }
}

```

```

}
class InterfacePrivateMethod implements Vehicle
{
    public static void main(String[] args)
    {
        InterfacePrivateMethod car = new InterfacePrivateMethod();
        car.start();
        car.stop();
    }
}

```

OUTPUT

```

Vehicle is Starting
Vehicle is Stopping

```

The `log()` method cannot be directly called from the implementing class.

Functional Interface

A Functional Interface is an interface with a single abstract method (SAM). It is used as the basis for lambda expressions and method references introduced in Java 8. Annotated with `@FunctionalInterface` (optional but recommended) to enforce the rule at compile time.

We can implement it using a lambda expression:

Example: 1

```

interface Calculator
{
    int add(int a, int b);
}
public class FunctionalInterface
{
    public static void main(String[] args)
    {
        Calculator c = (a, b) -> a + b;
        System.out.println("Sum = " + c.add(10, 20));
    }
}

```

```
    }
}
```

OUTPUT

Sum = 30

Common Functional Interfaces

Java provides many functional interfaces in:

`java.util.function`

Some important ones are:

Interface	Purpose
Predicate<T>	Tests a condition
Function<T,R>	Converts one value to another
Consumer<T>	Consumes a value
Supplier<T>	Supplies a value
UnaryOperator<T>	Operates on one value
BinaryOperator<T>	Operates on two values

Example with Predicate:

```
import java.util.function.Predicate;
public class PredicateInterface
{
    public static void main(String[] args)
    {
        Predicate<Integer> p = n -> n > 10;
        System.out.println(p.test(20));
        System.out.println(p.test(5));
    }
}
```

OUTPUT

```
true
false
```

Interface with Lambda Expression

A lambda expression can be used with a functional interface.

```
//@FunctionalInterface
interface Greeting
{
    void sayHello(String name);
}

public class InterfaceLambda
{
    public static void main(String[] args)
    {
        Greeting g = name -> System.out.println("Hello " + name);
        g.sayHello("Imran");
    }
}
```

OUTPUT
Hello Imran

Interface and Abstract Class

Students often confuse interfaces with abstract classes.

Abstract class

```
abstract class Animal
{
    abstract void sound();
    void eat()
    {
        System.out.println("Animal eats");
    }
}
```

Interface

```
interface Animal
{
    void sound();
    default void eat()
    {
        System.out.println("Animal eats");
    }
}
```

Major Differences:

Feature	Interface	Abstract Class
Keyword	interface	abstract class
Object creation	No	No, if abstract
Multiple inheritance	Multiple interfaces can be implemented	A class can extend only one class
Constructors	No	Yes
Instance variables	No ordinary instance fields; interface fields are public static final	Yes
Abstract methods	Yes	Yes
Concrete methods	default, static, private methods can have bodies	Yes
Access modifiers for abstract methods	Public contract	Can use various access levels
Static methods	Yes	Yes
Final methods	Not applicable to abstract interface methods	Yes

Interface vs Class

Interface	Class
Declared using interface	Declared using class
Cannot normally be instantiated directly	Can be instantiated if concrete
Used to define a contract	Used to define state and behaviour
A class uses implements	A class uses extends to inherit
Supports multiple interfaces	Class can extend only one class
Fields are public static final	Fields can have different modifiers
Constructors are not allowed	Constructors are allowed

Multiple Interfaces with Same Method

```

interface A
{
    void show();
}
interface B
{
    void show();
}
A class can implement both:
class Test implements A, B
{
    @Override
    public void show()
    {
        System.out.println("Hello");
    }
}

```

This causes no problem because both interfaces require the same method signature.

Conflict Between Default Methods

Consider:

```
interface A
{
    default void show()
    {
        System.out.println("A");
    }
}
interface B
{
    default void show()
    {
        System.out.println("B");
    }
}
```

Now:

```
class Test implements A, B
{
}
```

This creates a conflict.

Java does not know whether to use **A.show()** or **B.show()**.

Therefore, the class must override the method:

```
class Test implements A, B
{
    @Override
    public void show()
    {
        System.out.println("Test");
    }
}
```

Interface Inheritance Example

```
interface Person
{
    void walk();
}

interface Employee extends Person
{
    void work();
}

interface Manager extends Employee
{
    void manage();
}

class ProjectManager implements Manager
{
    @Override
    public void walk()
    {
        System.out.println("Manager walks");
    }
    @Override
    public void work()
```

```

    {
        System.out.println("Manager works");
    }
    @Override
    public void manage()
    {
        System.out.println("Manager manages project");
    }
}
public class InterfaceInheritance1
{
    public static void main(String[] args)
    {
        ProjectManager pm = new ProjectManager();
        pm.walk();
        pm.work();
        pm.manage();
    }
}

```

Interface and Loose Coupling

Consider:

```

class OrderService
{
    private UPIPayment payment = new UPIPayment();
}

```

This creates tight coupling.

Instead:

```
class OrderService
{
    private Payment payment;

    public OrderService(Payment payment)
    {
        this.payment = payment;
    }

    public void makePayment(double amount)
    {
        payment.pay(amount);
    }
}
```

Now:

```
public class Main
{
    public static void main(String[] args)
    {
        Payment payment = new UPIPayment();
        OrderService order = new OrderService(payment);
        order.makePayment(5000);
    }
}
```

Later, we can change:

```
Payment payment = new CardPayment();
```

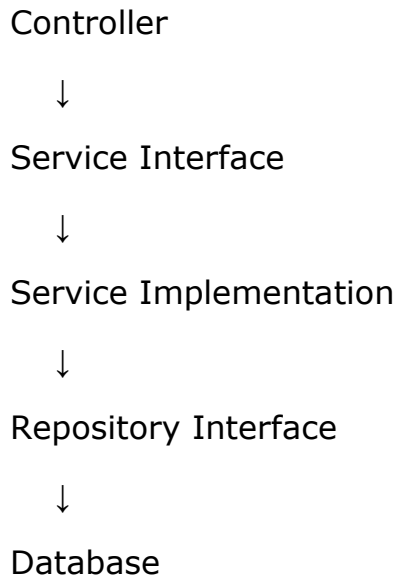
without changing `OrderService`.

This is a fundamental concept in **Spring and Spring Boot dependency injection**.

Interface in Spring Boot

Interfaces are heavily used in Spring applications.

Typical architecture:



Example:

```
public interface StudentService
{
    void addStudent(Student student);
    Student getStudent(Long id);
    void deleteStudent(Long id);
}
```

Implementation:

```
@Service
public class StudentServiceImpl implements StudentService
{
    @Override
    public void addStudent(Student student)
```

```
{  
    System.out.println("Student added");  
}  
  
@Override  
public Student getStudent(Long id)  
{  
    return null;  
}  
  
@Override  
public void deleteStudent(Long id)  
{  
    System.out.println("Student deleted");  
}  
}
```

The controller can depend on the interface:

```
@RestController  
public class StudentController  
{  
    private final StudentService service;  
    public StudentController(StudentService service)  
    {  
        this.service = service;  
    }  
}
```

NOTE: This is one reason interfaces are extremely important in Java Full Stack development.

Practice Programs for Students

After learning interfaces, students should practice these programs:

Beginner

- ✓ Create a Shape interface with **area()**.
- ✓ Implement it using Circle.
- ✓ Implement it using Rectangle.
- ✓ Create a Vehicle interface with **start()** and **stop()**.
- ✓ Create Car and Bike implementations.

Intermediate

- ✓ Create **Payment** interface with UPI, Card and Net Banking implementations.
- ✓ Create **Employee** and **Manager** interfaces.
- ✓ Demonstrate multiple interface implementation.
- ✓ Demonstrate interface inheritance.
- ✓ Demonstrate default methods.

Advanced

- ✓ Demonstrate conflicting default methods.
- ✓ Create a functional interface and use a lambda expression.
- ✓ Use Predicate, Function, Consumer, and Supplier.
- ✓ Create a small banking application using interfaces.
- ✓ Create a Spring Boot application using a service interface and implementation.

Final Conclusion

An interface in Java is a contract between the interface and the implementing class. It specifies what operations a class should provide while allowing the class to decide how those operations are implemented.

The most important concepts to remember are:

Interface
↓
Contract
↓
implements
↓
Implementation
↓
Polymorphism
↓
Loose Coupling

For a Java Full Stack developer, interfaces are particularly important because the same concepts appear throughout **collections, functional programming, JDBC, Spring, Spring Boot, dependency injection, service layers, repositories**, and **API** design.